



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

پیاده‌سازی حافظه شرکت پذیر نوع سه گانه

نام درس

آزمایشگاه طراحی سامانه‌های رقمی

تابستان ۱۴۰۵

دستیار آموزشی

علی اثنی‌عشری

گروه ۲۰

علی سلطانی - ۴۰۳۱۰۶۰۹۲

یگانه کربلایی - ۴۰۳۱۰۶۵۲۷

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	هدف آزمایش	۲
۲	مبانی نظری	۲
۳	طراحی و الگوریتم پیاده‌سازی	۲
۱.۳	نمایش داده سه‌مقداری	۲
۲.۳	بیت اعتبار (Valid)	۳
۳.۳	نمودار بلوکی پیمان	۳
۴.۳	منطق نوشتن (برنامه‌ریزی ثبات‌ها)	۳
۵.۳	منطق مقایسه و انکودر اولویت	۳
۴	پیاده‌سازی Verilog	۳
۱.۴	پارامتریک بودن طراحی	۴
۵	شبیه‌سازی و تست	۴
۱.۵	سناریوهای آزمون	۴
۲.۵	نتیجه موج شبیه‌سازی	۵
۶	سنتز و بررسی منابع مصرفی روی FPGA	۵
۷	ملاحظات پیاده‌سازی روی برد FPGA	۶
۸	نتیجه‌گیری	۶

۱ هدف آزمایش

هدف این آزمایش، طراحی و پیاده‌سازی یک پیمانه حافظه شرکت‌پذیر نوع سه گانه^۱ موسوم به TCAM است. این پیمانه باید شامل ۱۶ ثبات ۱۶ بیتی باشد به گونه‌ای که هر بیت از هر ثبات بتواند یکی از سه مقدار 0، 1 یا X^۲ را ذخیره کند. طراحی باید کاملاً پارامتریک بوده و برای سنتز روی پورده FPGA مناسب باشد. علاوه بر پیاده‌سازی منطق ذخیره‌سازی و مقایسه، صحت عملکرد پیمانه باید با استفاده از یک فایل آزمون^۳ در نرم‌افزار ModelSim بررسی شود و گزارش کامل مراحل طراحی، تصمیمات اتخاذ شده و نتایج شبیه‌سازی ارائه گردد.

۲ مبانی نظری

حافظه شرکت‌پذیر^۴ برخلاف حافظه‌های معمولی که با آدرس به محتوا دسترسی پیدا می‌کنند، در جهت عکس عمل می‌کند: یک الگوی جستجو به آن داده می‌شود و پیمانه مشخص می‌کند که آیا این الگو در میان محتوای ذخیره‌شده وجود دارد یا خیر، و در صورت وجود، آدرس (یا آدرس‌های) محل آن را برمی‌گرداند. این نوع حافظه به‌طور گسترده در کاربردهایی مانند جدول مسیریابی شبکه، حافظه‌های نهان^۵، فشرده‌سازی داده، بانک داده و سامانه‌های هوشمند به کار می‌رود.

حافظه شرکت‌پذیر نوع سه گانه^۶ تعمیمی از CAM معمولی است. تفاوت اصلی آن با CAM عادی این است که علاوه بر مقادیر 0 و 1، هر بیت از هر ثبات می‌تواند مقدار X نیز داشته باشد. به این معنا که در فرایند مقایسه و انطباق^۷، بیت‌های X نادیده گرفته می‌شوند و صرف‌نظر از مقدار بیت متناظر در الگوی جستجو، همواره با آن منطبق در نظر گرفته می‌شوند. برای مثال، اگر داده 01101110 در یک TCAM جستجو شود، با هر یک از الگوهای زیر که در ثبات‌های آن ذخیره شده باشند به عنوان انطباق شناسایی می‌شود:

XXXXXXXX 0X1X11X0 X1101XXX 0110XXXX

این ویژگی، TCAM را برای کاربردهایی مانند طبقه‌بندی بسته‌های شبکه و جدول‌های مسیریابی با پیشوند متغیر^۸ بسیار مناسب می‌کند، چرا که یک قانون واحد می‌تواند به‌طور هم‌زمان بازه‌ای از آدرس‌ها را پوشش دهد.

۳ طراحی و الگوریتم پیاده‌سازی

۱.۳ نمایش داده سه‌مقداری

از آنجا که سخت‌افزارهای رقمی معمولی تنها دو حالت 0 و 1 را پشتیبانی می‌کنند، برای نمایش حالت سوم (X) از یک نمایش دو کلمه‌ای استفاده شده است. برای هر ثبات، دو کلمه ۱۶ بیتی نگهداری می‌شود:

data: مقدار 0 یا 1 هر بیت را نگه می‌دارد.

mask: تعیین می‌کند که آیا آن بیت باید در مقایسه شرکت داده شود (1) یا فاقد اهمیت^۹ است (0).

با این نمایش، رابطه انطباق برای بیت i از ثبات j به صورت زیر تعریف می‌شود:

$$\text{term}_i(j) = \overline{\text{mask}_i(j)} \vee \text{data}_i(j) \oplus \text{search}(j)$$

یعنی اگر آن بیت باشد ($\text{mask}_i(j) = 0$)، عبارت همواره درست است؛ در غیر این صورت باید بیت داده دقیقاً با بیت الگوی جستجو برابر باشد. برای این که ثبات j به‌طور کامل با الگوی جستجو منطبق شناخته شود، باید تمام ۱۶ بیت آن شرط را برآورده کنند:

$$\text{match}_i = \bigwedge_{j=0}^{15} \text{term}_i(j)$$

¹Ternary Content Addressable Memory

²Don't Care

³Testbench

⁴Content Addressable Memory-CAM

⁵Cache

⁶Ternary

⁷Match

⁸Longest Prefix Match

⁹Don't Care

۲.۳ بیت اعتبار (Valid)

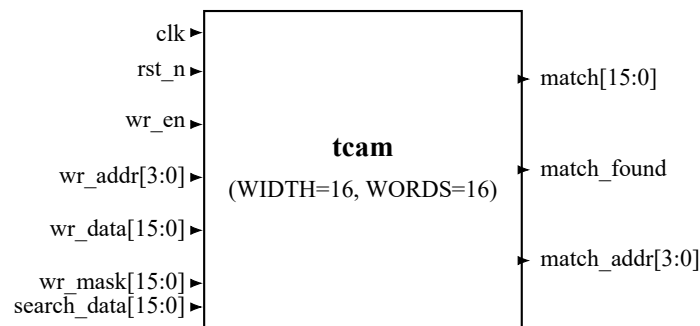
یکی از نکات ظریفی که در حین طراحی و شبیه‌سازی مشخص شد، رفتار ثبات‌های هنوز برنامه‌ریزی نشده است. اگر پس از ریست، کلمه mask تمام ثبات‌ها صفر باشد، این به معنای آن است که تمام بیت‌های آن‌ها Don't Care هستند؛ در نتیجه چنین ثباتی با هر الگوی جستجویی منطبق می‌شود که کاملاً نادرست و گمراه‌کننده است. برای رفع این مشکل، یک بیت مستقل به نام valid برای هر ثبات اضافه شد که تنها پس از نوشتن واقعی در آن آدرس به مقدار یک تنظیم می‌شود. رابطه نهایی انطباق به صورت زیر اصلاح شد:

$$match_i = valid_i \wedge \bigwedge_{j=0}^{15} term_i(j)$$

این تغییر با شبیه‌سازی نیز تأیید شد: در سناریوی آزمون شماره ۵ (بخش ۵)، یک الگوی کاملاً تصادفی جستجو شد و مشاهده شد که تنها ثبات از پیش برنامه‌ریزی شده (ثبات ۳ که کاملاً Don't Care تعریف شده بود) با آن منطبق می‌شود، نه ثبات‌های نانوشته ۴ تا ۱۵.

۳.۳ نمودار بلوکی پیمانه

شکل ۱: نمای کلی ورودی‌ها و خروجی‌های پیمانه TCAM طراحی شده را نشان می‌دهد.



شکل ۱: نمودار بلوکی پیمانه tcam همراه با پورت‌های ورودی و خروجی

۴.۳ منطق نوشتن (برنامه‌ریزی ثبات‌ها)

نوشتن در پیمانه به صورت همزمان^{۱۰} و روی لبه بالارونده ساعت^{۱۱} انجام می‌شود. با فعال بودن سیگنال wr_en، مقادیر wr_data و wr_mask در آدرس wr_addr نوشته شده و بیت valid متناظر یک می‌شود. با فعال شدن سیگنال ریست (rst_n) با مقدار پایین، تمام ثبات‌ها به حالت نامعتبر بازمی‌گردند.

۵.۳ منطق مقایسه و انکودر اولویت

منطق مقایسه به صورت کاملاً ترکیبی^{۱۲} و با استفاده از حلقه generate برای هر ۱۶ ثبات پیاده‌سازی شده است؛ به این ترتیب طراحی نسبت به پارامترهای WIDTH و WORDS به طور کامل مقیاس پذیر است. در صورتی که بیش از یک ثبات با الگوی جستجو منطبق شود (که در TCAM کاملاً محتمل و بخشی از کاربرد آن است)، خروجی match_addr با استفاده از یک تابع انکودر اولویت، آدرس اولین ثبات (کمترین زیروند) منطق را گزارش می‌کند. خروجی match_found نیز به سادگی OR منطقی تمام بیت‌های بردار match است.

۴ پیاده‌سازی Verilog

پیمانه به طور کامل با زبان Verilog و به صورت پارامتریک (با پارامترهای WIDTH و WORDS) نوشته شده است. فایل کامل طراحی (tcam.v) و فایل آزمون (tcam_tb.v) به طور کامل همراه این گزارش ارسال شده‌اند. در ادامه، هسته اصلی منطق مقایسه و انکودر اولویت (که مهم‌ترین بخش الگوریتمی طراحی است) آورده شده است:

¹⁰Synchronous

¹¹Clock

¹²Combinational

Listing 1: Core match logic and priority encoder (excerpt from tcam.v)

```

1 genvar g;
2 generate
3   for (g = 0; g < WORDS; g = g + 1) begin : gen_match
4     assign match[g] = valid_mem[g] &
5       ( &( ~mask_mem[g] | ~(data_mem[g] ^ search_data) ) );
6   end
7 endgenerate
8
9 assign match_found = |match;
10
11 function [ADDR_W-1:0] priority_encode;
12   input [WORDS-1:0] m;
13   integer k;
14   begin
15     priority_encode = {ADDR_W{1'b0}};
16     for (k = WORDS-1; k >= 0; k = k - 1)
17       if (m[k]) priority_encode = k[ADDR_W-1:0];
18   end
19 endfunction
20
21 assign match_addr = priority_encode(match);

```

۱.۴ پارامتریک بودن طراحی

پارامترهای WIDTH (عرض هر ثبات، مقدار پیش‌فرض ۱۶) و WORDS (تعداد ثبات‌ها، مقدار پیش‌فرض ۱۶) در ابتدای پیمانه تعریف شده‌اند و عرض باس آدرس (ADDR_W) نیز به‌طور خودکار با استفاده از تابع $\$clog2$ از روی WORDS محاسبه می‌شود. به این ترتیب، برای تغییر ابعاد پیمانه (مثلاً به ۳۲ ثبات ۳۲ بیتی) تنها کافی است مقدار این دو پارامتر در نمونه‌سازی پیمانه تغییر کند و نیازی به تغییر کد داخلی نیست.

۵ شبیه‌سازی و تست

۱.۵ سناریوهای آزمون

برای اطمینان از صحت عملکرد پیمانه، یک فایل آزمون در ModelSim نوشته شد که پنج سناریوی زیر را پوشش می‌دهد. ابتدا چهار ثبات با ماسک‌های متفاوت برنامه‌ریزی می‌شوند (جدول ۱)، سپس پنج الگوی جستجو اعمال می‌شود (جدول ۲):

جدول ۱: ثبات‌های برنامه‌ریزی شده در فایل آزمون

آدرس	data	mask	توضیح
۰	0x006E	0x00FF	الگوی کاملاً مشخص (0110_1110)، بدون X
۱	0x0060	0x00F0	الگوی 0110XXXX (نیمه پایین X)
۲	0x000E	0x000F	الگوی XXXX1110 (نیمه بالا X)
۳	0x0000	0x0000	کاملاً X — با هر مقداری منطبق می‌شود

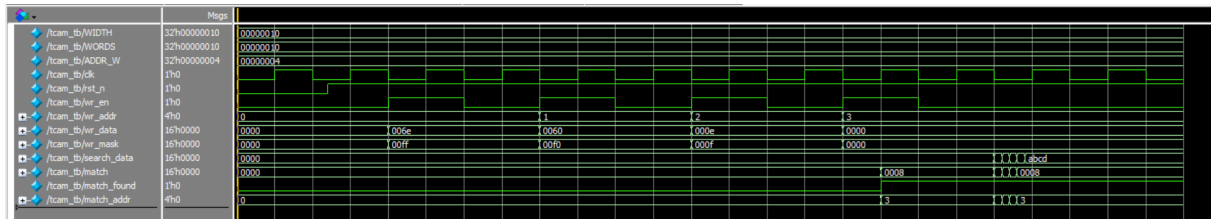
جدول ۲: نتایج پنج سناریوی جستجو (خروجی تأییدشده در شبیه‌سازی)

تست	الگوی جستجو (search_data)	match (باینری)	match_addr
۱	0x006E — تطابق دقیق با ثبات ۰	0000000000001111	۰
۲	0x0065 — تست نیمه پایین X	0000000000001010	۱
۳	0x000E — تست نیمه بالا X	0000000000001100	۲
۴	0x00FF — فقط باید با ثبات کاملاً X منطبق شود	0000000000001000	۳
۵	0xABCD — الگوی کاملاً تصادفی (بررسی ثبات‌های نانوشته)	0000000000001000	۳

نکته مهم در تست شماره ۵: با وجود آنکه الگوی جستجو کاملاً دلخواه و بی‌ارتباط با محتوای برنامه‌ریزی شده است، بردار match تنها بیت مربوط به ثبات ۳ (که عمداً کاملاً دون‌کر تعریف شده) را يك نشان می‌دهد و بیت‌های مربوط به ثبات‌های ۴ تا ۱۵ (که هرگز نوشته نشده‌اند) صفر باقی می‌مانند. این دقیقاً همان رفتاری است که با افزودن بیت valid انتظار می‌رفت و صحت این تصمیم طراحی را تأیید می‌کند.

۲.۵ نتیجه موج شبیه‌سازی

شکل ۲ خروجی پنجره موج ModelSim پس از اجرای کامل فایل آزمون را نشان می‌دهد. در بازه ابتدایی (تا نانوثانیه ۴۰ام) چهار عملیات نوشتن روی ثبات‌های ۰ تا ۳ قابل مشاهده است (تغییرات wr_data, wr_addr, wr_mask و در ادامه، برای هر يك از پنج الگوی جستجو، تغییر متناظر در match_found, match_addr و match دیده می‌شود که کاملاً با مقادیر جدول ۲ مطابقت دارد.



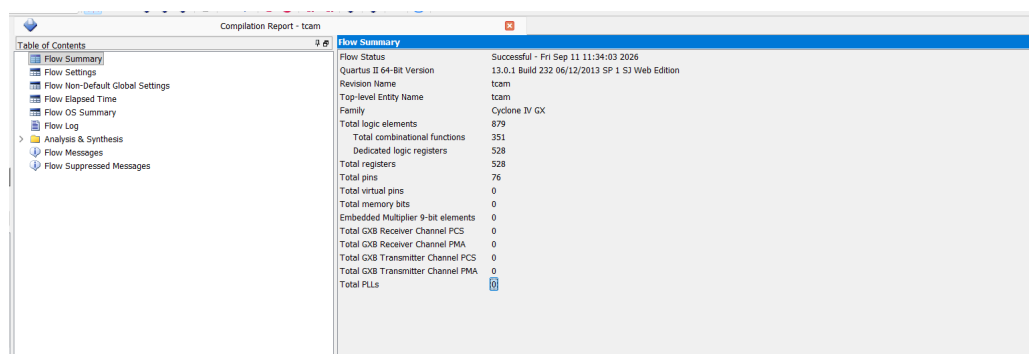
شکل ۲: پنجره موج ModelSim پس از اجرای فایل آزمون — عملیات نوشتن چهار ثبات و نتیجه پنج جستجو

۶ سنتز و بررسی منابع مصرفی روی FPGA

پس از تأیید صحت عملکرد در شبیه‌سازی، طراحی در نرم‌افزار Quartus با پیمانه tcam به‌عنوان Top-Level Entity سنتز شد. خلاصه مصرف منابع گزارش شده توسط Analysis & Synthesis در جدول ۳ آمده است.

جدول ۳: خلاصه مصرف منابع منطقی پس از سنتز (Analysis & Synthesis Summary)

مقدار	پارامتر
879 (351 Combinational)	تعداد عناصر منطقی
528	تعداد ثبات‌ها
N/A	حداکثر بسامد کاری تخمینی



شکل ۳: خلاصه گزارش سنتز در Quartus

از آنجا که طراحی برای ابعاد ۱۶ ثبات ۱۶ بیتی ($16 \times 16 = 256$) سلول سه‌مقداری (پیاده‌سازی شده، مصرف منابع منطقی (LUT و ثبات) در حد معقولی باقی می‌ماند و مشکلی برای سنتز روی برد آموزشی ایجاد نمی‌کند. لازم به ذکر است که این پیاده‌سازی کاملاً مبتنی بر ثبات و منطق ترکیبی (Register + LUT based) است، در حالی که پیاده‌سازی‌های صنعتی واقعی TCAM معمولاً از حافظه‌های BRAM یا سلول‌های حافظه اختصاصی سه‌مقداری استفاده می‌کنند تا در ابعاد بزرگ (هزاران ورودی) از مصرف بیش از حد منابع LUT جلوگیری شود. برای ابعاد کوچک این آزمایش، رویکرد مبتنی بر منطق ترکیبی انتخاب مناسب و کافی است.

۷ ملاحظات پیاده‌سازی روی بورد FPGA

یکی از محدودیت‌های عملی پیاده‌سازی این طرح روی بورد FPGA، تعداد ورودی‌های مورد نیاز است. برای بهره‌گیری کامل از پیمان، باید بتوان `wr_addr` (۴ بیت)، `wr_data` (۱۶ بیت)، `wr_mask` (۱۶ بیت) و `search_data` (۱۶ بیت) را از طریق سویچ‌های فیزیکی بورد وارد کرد که مجموعاً ۵۲ بیت است.

برای رفع این محدودیت، ورودی به‌صورت مرحله‌ای وارد خواهد شد: با استفاده از چند سویچ برای انتخاب حالت (نوشتن آدرس، نوشتن داده، نوشتن ماسک یا جستجو) و یک ثبات شیفت داخلی که با فشردن یک دکمه، مقادیر سویچ‌های ورودی را به‌تدریج در یک ثبات میانی بارگذاری می‌کند، امکان وارد کردن مقادیر چندبیتی با تعداد محدودی سویچ فراهم می‌شود. خروجی `match_found` و `match_addr` نیز با توجه به تعداد LED و نمایشگر هفت‌قسمتی موجود روی بورد، به‌صورت چندمرحله‌ای یا با کدگذاری Hex روی نمایشگر نمایش داده خواهد شد.

۸ نتیجه‌گیری

در این آزمایش، یک پیمان حافظه شرکت‌پذیر سه‌گانه^{۱۳} با ۱۶ ثبات ۱۶ بیتی و به‌صورت کاملاً پارامتریک طراحی و پیاده‌سازی شد. نمایش سه‌مقداری هر بیت با استفاده از دو کلمه `data` و `mask` انجام شد و برای رفع مشکل انطباق نادرست ثبات‌های نانوشته، بیت اعتبار^{۱۴} به طراحی اضافه گردید. تصمیمی که با شبیه‌سازی مورد تأیید قرار گرفت. عملکرد طرح با پنج سناریوی آزمون مختلف در ModelSim بررسی و صحت کامل آن (شامل تطابق دقیق، تطابق با بیت‌های X در نیمه‌های مختلف کلمه، و رفتار صحیح ثبات‌های نانوشته) تأیید شد. طراحی سپس در Quartus با موفقیت سنتز شد. در نهایت، محدودیت تعداد ورودی‌های فیزیکی بورد در برابر عرض ۱۶ بیتی داده‌ها بررسی و راهکار ورودی مرحله‌ای برای پیاده‌سازی عملی روی بورد پیشنهاد شد.

¹³TCAM

¹⁴valid